Table-Based Encodings for Conway's Doomsday Algorithm: Vectorized Doomsdays and Doomyears

Thomas Wollin
Toronto, Ontario, Canada
github.com/Nillows

March 28, 2026

Abstract

Conway's Doomsday Algorithm (1973) determines the day of the week for any date in the Gregorian calendar via three additive components: a century anchor, a year offset, and a month–day offset. The century anchor is a fixed four-entry table. The other two components require live arithmetic: the year offset demands computing $y + \lfloor y/4 \rfloor \pmod 7$, and the month–day offset requires a subtraction that can produce negative intermediate values. We present two new encoding schemes that replace both arithmetic steps with structured table lookups.

The first, *vectorized doomsdays*, re-encodes each month's doomsday date as a two-digit number whose tens and units digits represent the backward and forward gaps (respectively) from the nearest multiples-of-seven month anchors. A directional crossing rule (the "square knot rule") pairs the target date's gap with the opposite-direction digit, reducing the month–day offset to a single-digit addition.

The second, *Doomyears*, encodes the year-offset function as a navigational lookup exploiting the 28-year periodicity of the Gregorian weekday cycle.

Together with Conway's century anchor table, these form a unified system we call the *Calamity Tables*. We prove correctness, establish self-verification properties, analyse the internal structure of both encodings, and compare the cognitive complexity of the Calamity Table system against the standard arithmetic method.

1 Introduction

The Doomsday Algorithm, introduced by Conway [3] and described in [1], computes the day of the week for any date as:

$$\text{day of week} \equiv c + \omega(y) + \delta(m, d) \pmod 7, \tag{1}$$

where c is the century anchor, $\omega(y) = (y + \lfloor y/4 \rfloor) \bmod 7$ is the year offset, and $\delta(m, d)$ is the signed distance from the target date to the month's doomsday date.

Step c is a four-entry table. Steps ω and δ require real-time arithmetic. We replace both with pre-computed encodings. The three tables — century anchors, vectorized doomsdays, and Doomyears — we collectively term the *Calamity Tables*, in keeping with the eschatological naming tradition of the algorithm they extend.

2 Part 1: Vectorized doomsdays

2.1 Month anchors and gap structure

Definition 1. The *month anchors* are $A = \{7, 14, 21, 28\}$, the multiples of 7 occurring as valid dates in every month.

Definition 2. For a date t with $7 \leq t \leq 31$, let $a^-(t) = \max\{a \in A : a \leq t\}$ and $a^+(t) = \min\{a \in A : a \geq t\}$. Define the *forward gap* $g_F(t) = t - a^-(t)$ and the *backward gap* $g_B(t) = a^+(t) - t$. For dates $1 \leq t \leq 6$, we set $a^+(t) = 7$ and $g_B(t) = 7 - t$, with $g_F(t)$ undefined (no valid anchor below). For dates $29 \leq t \leq 31$, we have $a^-(t) = 28$ and $g_F(t) = t - 28$, with $g_B(t)$ undefined (no valid anchor in A at or above t). In both boundary cases the square knot rule must use the direction that is defined. For the purpose of constructing V_m, the undefined gap is taken as 7 minus the defined gap, extending the complementarity $g_F + g_B = 7$ to all dates $1 \leq t \leq 31$ not on an anchor.

Proposition 3. *For any date t with $7 \leq t \leq 28$ not on an anchor, $g_F(t) + g_B(t) = 7$. When $t \in A$, $g_F(t) = g_B(t) = 0$.*

Proof. If t lies strictly between consecutive anchors a^- and a^+, then $a^+ - a^- = 7$, so $g_F + g_B = (t - a^-) + (a^+ - t) = 7$. If $t \in A$, then $a^- = a^+ = t$. □

2.2 Construction of the vectorized doomsdays

Definition 4. Let d_m denote the traditional doomsday date for month m. The *vectorized doomsday* for month m is

$$V_m = 10 \cdot g_B(d_m) + g_F(d_m).$$

The explicit values are:

Month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
d_m	3	28	7	4	9	6	11	8	5	10	7	12
V_m	43	00	00	34	52	16	34	61	25	43	00	25

Table 1: Traditional doomsday dates and vectorized doomsdays. Leap year: Jan $d_m = 4 \to V = 34$; Feb $d_m = 29 \to V = 61$.

Proposition 5 (Digit-sum property)**.** *For every vectorized doomsday V_m, either both digits are 0 or the digits sum to 7.*

Proof. Immediate from Proposition 3: the tens digit is $g_B(d_m)$ and the units digit is $g_F(d_m)$. □

Proposition 6 (Spatial encoding)**.** *The tens (left) digit of V_m encodes the leftward (backward) gap; the units (right) digit encodes the rightward (forward) gap. The direction a digit represents matches the direction one reads to find it.*

2.3 The square knot rule

Theorem 7 (Correctness of the month–day offset)**.** *Let t be a target date in month m, with forward gap $g_F(t)$ from its nearest lower anchor. Let $V_m = 10b + f$ where $b = g_B(d_m)$ and $f = g_F(d_m)$. Then*

$$t - d_m \equiv g_F(t) + b \pmod 7.$$

Similarly, if the backward gap $g_B(t)$ is used instead, then

$$d_m - t \equiv g_B(t) + f \pmod 7.$$

Proof. Write $t = a^-(t) + g_F(t)$ where $a^-(t)$ is a multiple of 7. Write $d_m = a^+(d_m) - g_B(d_m) = a^+(d_m) - b$ where $a^+(d_m)$ is a multiple of 7. Then $t - d_m = a^-(t) + g_F(t) - a^+(d_m) + b \equiv g_F(t) + b \pmod 7$, since both $a^-(t)$ and $a^+(d_m)$ are multiples of 7.

The second identity follows by symmetry: $d_m - t \equiv g_B(t) + f \pmod 7$ by the same argument with roles exchanged. □

Remark 8. The forward-gap case gives a result in the context of "days after doomsday" (added to the year's doomsday), while the backward-gap case gives "days before doomsday" (subtracted). The choice of direction at the target date determines both which digit is selected *and* the sign of the result — these are not independent decisions.

Remark 9 (Why same-direction pairing requires subtraction)*.* One might ask whether the encoding could instead pair same directions — forward with forward, backward with backward — eliminating the crossing rule. Same-direction pairing is valid, but only under *subtraction*: $g_F(t) - g_F(d_m) \equiv t - d_m \pmod 7$, since both reference anchors are multiples of 7 and cancel. Under *addition*, $g_F(t) + g_F(d_m) \equiv t + d_m \pmod 7$, which differs from the correct value $t - d_m$ by $2d_m$. One could reverse the digit positions in V_m to support same-direction subtraction, but this reintroduces the negative intermediate values that the encoding was designed to eliminate. The cross-direction rule is therefore the structural cost of an all-addition, all-non-negative system.

Corollary 10 (Self-consistency)**.** *For every month m, running the traditional doomsday date d_m through its own vectorized doomsday yields offset* 0*.*

Proof. $g_F(d_m) + g_B(d_m) = 7 \equiv 0 \pmod 7$ (or $0 + 0 = 0$ when d_m is an anchor). □

2.4 Structure of the code space

Proposition 11. *Exactly four distinct non-zero digit pairs appear among the vectorized doomsdays: $\{1, 6\}$, $\{2, 5\}$, $\{3, 4\}$, and $\{0, 0\}$. Each non-zero pair appears in both orientations (e.g.,* 16 *and* 61*).*

Proof. Since months have at most 31 days and the highest anchor is 28, the maximum forward gap from anchor 28 is $31 - 28 = 3$. Thus $g_F \in \{0, 1, 2, 3\}$ for any date in such a month. Since $g_F + g_B = 7$ (or both $= 0$), the only possible pairs (g_B, g_F) are $(0, 0)$, $(6, 1)$, $(5, 2)$, and $(4, 3)$ — that is, the unordered pairs $\{0, 0\}$, $\{1, 6\}$, $\{2, 5\}$, and $\{3, 4\}$. Inspection of Table 1 confirms that each of these four pairs is realised by at least one traditional doomsday date, and each non-zero pair appears in both orientations. □

2.5 Equivalence classes of anchor-date systems

The vectorized doomsday construction does not depend on Conway's specific choice of anchor dates. Any set of twelve dates (one per month) that share a common weekday within each year can serve as input, and the construction produces a valid code table. We now classify all such systems and show that exactly seven equivalence classes exist, that they are related by a single cyclic rotation of codes, and that Conway's class is uniquely optimal.

2.5.1 Classification

Definition 12. Two anchor-date systems $\{d_m\}$ and $\{d'_m\}$ are *equivalent* if $d_m \equiv d'_m \pmod 7$ for every month m.

Since the vectorized doomsday V_m depends only on $d_m \bmod 7$, equivalent systems produce identical code tables.

Proposition 13 (Seven equivalence classes)**.** *There are exactly* 7 *equivalence classes of same-weekday anchor-date systems, indexed by a uniform offset* $k \in \{0, 1, \ldots, 6\}$ *from Conway's doomsday dates. System* k *has anchor-date residues* $d_m + k \pmod 7$ *and century anchors* $c - k \pmod 7$*, where* c *is Conway's century anchor.*

Proof. Let $\{d_m\}$ be Conway's doomsday dates and $\{d'_m\}$ any other same-weekday system. In a given year, all d_m share a weekday W and all d'_m share a weekday W'. Within each month m, the weekday of the nth is determined by $n \bmod 7$ (up to a month-specific constant), so $d'_m \equiv d_m + (W' - W) \pmod 7$ for every m. Setting $k = W' - W \bmod 7$ gives a uniform shift. Since the overall algorithm must yield the same final weekday, the century anchor absorbs the complementary shift: $c' = c - k \pmod 7$. □

2.5.2 The rotation cycle

The seven classes are connected by a cyclic rotation of codes. To see why, observe how the vectorized doomsday map works at the level of residues. For a doomsday date with residue $r = d_m \bmod 7$, the forward gap is $g_F = r$ and the backward gap is $g_B = (7 - r) \bmod 7$, giving code $V = 10 \cdot g_B + g_F$. (When $r = 0$, both gaps are zero and $V = 00$.) Incrementing the residue by 1 — i.e., moving from class k to class $k + 1$ — maps each code to a specific successor.

Proposition 14 (Code rotation)**.** *The map* $r \mapsto r+1 \pmod 7$ *on residues induces the cyclic permutation of codes*

$$00 \to 61 \to 52 \to 43 \to 34 \to 25 \to 16 \to 00.$$

This is a single 7-cycle; no code is a fixed point.

Proof. At residue r, the code is $10(7-r)+r$ (for $r \neq 0$) or 00 (for $r = 0$). At residue $r+1$: the forward gap increases by 1 and the backward gap decreases by 1, giving $10(7-r-1)+(r+1) = 10(6-r)+r+1$. Tracing from $r = 0$: $00 \to 10(6)+1 = 61 \to 10(5)+2 = 52 \to 43 \to 34 \to 25 \to 16 \to 10(0)+7 = 10(0)+0 = 00$ (since at $r = 0$ both gaps reset to zero). The cycle has length 7, so every code participates. □

Remark 15. The rotation is *not* numerically obvious. One might expect incrementing the offset to produce simple numerical successors, but 00 is followed by 61, not by 11. The cycle reflects the underlying geometry: adding 1 to the forward gap while subtracting 1 from the backward gap simultaneously shifts both digit positions in opposite directions. The code vocabulary $\{00, 16, 25, 34, 43, 52, 61\}$ is the orbit of 00 under repeated application of this rotation.

2.5.3 Group-theoretic structure

The classification has a clean algebraic interpretation. The cyclic group $\mathbb{Z}/7\mathbb{Z}$ acts on the set of anchor-date systems by translation: element k sends system $\{d_m\}$ to system $\{d_m + k\}$. Since the code for month m depends only on d_m mod 7, this action descends to a permutation of the code table.

Proposition 16 (Invariance of month groupings and code vocabulary)**.** *All seven systems produce the same set of codes* $\{00, 16, 25, 34, 43, 52, 61\}$ *and preserve the same month groupings (which months share a code). The action of* $\mathbb{Z}/7\mathbb{Z}$ *permutes the codes across fixed month groups but never splits or merges groups.*

Proof. The code for month m in system k depends only on $(d_m + k)$ mod 7. The translation $k \mapsto k+1$ uniformly shifts every month's residue, applying the same cyclic permutation of codes (Proposition 14) to all months simultaneously. Since months with $d_m \equiv d_{m'} \pmod 7$ undergo the same shift, they always receive the same code — groupings are invariant. □

Remark 17. The code vocabulary $\{00, 16, 25, 34, 43, 52, 61\}$ is therefore an algebraic invariant of the Gregorian calendar itself, independent of any particular choice of anchor dates. The seven codes correspond to the seven elements of $\mathbb{Z}/7\mathbb{Z}$ under the bijection $r \mapsto 10 \cdot (-r \bmod 7) + r$, and the rotation cycle is simply the image of the generator $+1$ under this bijection. Choosing an anchor-date system is equivalent to choosing a labelling of the seven codes by $\mathbb{Z}/7\mathbb{Z}$; the structure of the code space is fixed.

2.5.4 Month groupings and their origin

The seven month groupings, determined by the residue classes of Conway's doomsday dates, are:

d_m mod 7	Months	Size	Code at $k = 0$
0	Feb, Mar, Nov	3	00
1	Aug	1	61
2	May	1	52
3	Jan, Oct	2	43
4	Apr, Jul	2	34
5	Sep, Dec	2	25
6	Jun	1	16

The group sizes $\{3, 2, 2, 2, 1, 1, 1\}$ are not arbitrary — they are determined by the Gregorian month-length sequence. The 12 doomsday dates are constrained to share a single weekday, so their residues mod 7 are fixed by the cumulative month-length offsets. Specifically, month lengths $(31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31)$ have residues $(3, 0, 3, 2, 3, 2, 3, 3, 2, 3, 2, 3)$ modulo 7. These accumulate across the year: each month's doomsday residue equals the previous month's residue plus the intervening month length, reduced mod 7. The resulting residue sequence $(3, 0, 0, 4, 2, 6, 4, 1, 5, 3, 0, 5)$ has the partition $\{3, 2, 2, 2, 1, 1, 1\}$ as a necessary consequence of twelve items distributed across seven bins by a fixed cumulative sum.

Remark 18. The partition $\{3, 2, 2, 2, 1, 1, 1\}$ is a structural fingerprint of the Gregorian calendar. A different month-length sequence (such as the Julian calendar before the Gregorian reform, or a hypothetical calendar with different month lengths) would produce a different residue distribution and therefore different group sizes. The fact that exactly one group has size 3 is what gives Conway's class its unique optimality — it is a consequence of the Gregorian month lengths, not of the vectorized doomsday encoding.

2.5.5 The complete code table for all seven classes

Table 2 displays the vectorized doomsday codes for every month under each of the seven equivalence classes. Each row is a rotation of the row above, with the rotation cycle from Proposition 14 applied uniformly across all months.

k	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	00-months	c_{2000}
0	43	00	00	34	52	16	34	61	25	43	00	25	Feb, Mar, Nov (3)	2
1	34	61	61	25	43	00	25	52	16	34	61	16	Jun (1)	1
2	25	52	52	16	34	61	16	43	00	25	52	00	Sep, Dec (2)	0
3	16	43	43	00	25	52	00	34	61	16	43	61	Apr, Jul (2)	6
4	00	34	34	61	16	43	61	25	52	00	34	52	Jan, Oct (2)	5
5	61	25	25	52	00	34	52	16	43	61	25	43	May (1)	4
6	52	16	16	43	61	25	43	00	34	52	16	34	Aug (1)	3

Table 2: Vectorized doomsday codes for all seven equivalence classes. Column c_{2000} gives the adjusted century anchor for the 2000s ($(2 - k)$ mod 7). Conway's system is $k = 0$; Wang's is $k = 5$. Each row is obtained from the previous by applying the rotation $00 \to 61 \to 52 \to 43 \to 34 \to 25 \to 16 \to 00$ to every entry.

Reading Table 2 column by column reveals the invariance of month groupings: February, March, and November always share a code (the same entry appears in all three columns

for every value of k), April and July always share a code, and so on. The *codes themselves* rotate from row to row, but the *grouping structure* is rigid.

2.5.6 Optimality of Conway's system

Proposition 19 (Optimality of Conway's system)**.** *Among the seven equivalence classes, Conway's ($k = 0$) is the unique class that places the zero code* 00 *on the largest month group (February, March, November — three months). Every other class places* 00 *on a group of size at most* 2*.*

Proof. The residue-0 group {Feb, Mar, Nov} has size 3; all other residue classes have size ≤ 2. Only $k = 0$ maps residue 0 to code 00. This is visible in the rightmost columns of Table 2: $k = 0$ is the only row with three zero-months. □

Remark 20. Code 00 is the most efficient to use: the month contributes nothing to the sum, so the month–day step reduces to a single gap measurement with no digit selection or addition. The number of zero-months is therefore a natural measure of system quality. Conway's class ($k = 0$) achieves three zero-months; classes $k = 2, 3, 4$ achieve two; and classes $k = 1, 5, 6$ achieve only one. The optimality of $k = 0$ is not a question of feasibility but of *efficiency*.

2.5.7 Known instances of the seven classes

Of the seven equivalence classes, two correspond to independently published anchor-date systems:

- **Conway ($k = 0$):** The original Doomsday Algorithm [3], using anchor dates Jan 3, Feb 28, Mar 7, Apr 4, May 9, Jun 6, Jul 11, Aug 8, Sep 5, Oct 10, Nov 7, Dec 12. Three zero-months.
- **Wang ($k = 5$):** The null-days algorithm [5], using anchor dates 1/1, 2/12, 3/5, 4/2, 5/7, 6/4, 7/9, 8/6, 9/3, 10/8, 11/12, 12/10. Every Wang date satisfies $d'_m \equiv d_m + 5 \pmod 7$, and the resulting code table is row $k = 5$ of Table 2: $61, 25, 25, 52, 00, 34, 52, 16, 43, 61, 25, 43$. One zero-month (May). Century anchors are shifted by -5: the 2000s anchor becomes $(2 - 5) \bmod 7 = 4$.

The remaining five classes ($k = 1, 2, 3, 4, 6$) do not appear to correspond to any previously published systems, though each admits valid anchor dates and a fully functional algorithm. Class $k = 2$ (zero-months: September and December) and class $k = 3$ (zero-months: April and July) tie for second-best after Conway's.

Remark 21. The classification shows that any future proposal for a "new" set of same-weekday anchor dates must fall into one of these seven classes. If it shares residues with an existing system, the resulting vectorized codes — and therefore the practitioner's experience of the month–day step — will be identical. Novelty in anchor-date systems, from the perspective of the vectorized encoding, is confined to at most seven possibilities.

3 Part 2: Doomyears

3.1 Periodicity

Theorem 22 (28-year periodicity)**.** *$\omega(y+28) = \omega(y)$ for all $y \geq 0$.*

Proof. $\lfloor (y+28)/4 \rfloor = \lfloor y/4 \rfloor + 7$, so the total increases by $28 + 7 = 35 \equiv 0 \pmod 7$. □

Corollary 23. *The anchor years $0, 28, 56, 84$ satisfy $\omega(y) = 0$.*

3.2 Packed-number sequences

Definition 24. For $d \in \{0, \ldots, 15\}$, define $f(d) = \omega(d)$, $b(d) = \omega(28 - d)$, and the packed numbers $F_d = 10d + f(d)$, $B_d = 10d + b(d)$.

Definition 25. The *Doomyear* at distance d is $D_d = 100d + 10 \cdot b(d) + f(d)$, encoding both directions in a single number: distance, backward digit, forward digit.

d	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
F_d	00	11	22	33	45	56	60	71	83	94	105	116	121	132	143	154
B_d	00	15	24	33	42	50	66	75	84	92	101	110	126	134	143	152
D_d	00	151	242	333	425	506	660	751	843	924	1015	1106	1261	1342	1433	1524

Table 3: Forward, backward, and interleaved Doomyear sequences.

3.3 Structural properties

Proposition 26 (Leap-year stumble)**.** *$\Delta_f(d) = f(d) - f(d-1) \pmod 7$ equals 2 when $4 \mid d$ and 1 otherwise.*

Proof. $\Delta_f(d) = 1 + \lfloor d/4 \rfloor - \lfloor (d-1)/4 \rfloor$. The floor difference is 1 when $4 \mid d$, else 0. □

Proposition 27 (Complementarity)**.** *For all $d \in \{0, \ldots, 15\}$,*

$$f(d) + b(d) \equiv \begin{cases} 0 \pmod 7 & \text{if } 4 \mid d, \\ 6 \pmod 7 & \text{otherwise.} \end{cases}$$

Proof. $f(d) + b(d) = d + \lfloor d/4 \rfloor + (28 - d) + \lfloor (28-d)/4 \rfloor \pmod 7 = 28 + \lfloor d/4 \rfloor + \lfloor (28-d)/4 \rfloor \pmod 7$. Writing $d = 4q + r$: when $r = 0$, $\lfloor d/4 \rfloor + \lfloor (28-d)/4 \rfloor = q + (7 - q) = 7 \equiv 0$; when $r > 0$, $= q + (6 - q) = 6$. Since $28 \equiv 0$, the result is 0 or 6. □

Remark 28. Proposition 27 shows that the Doomyear digit-sum property is *not* identical to that of the vectorized doomsdays. The month codes always have digits summing to 7 (or both 0), a consequence of the fixed seven-unit span between consecutive anchors. The Doomyear directional digits sum to $0 \pmod 7$ (i.e. both 0 or summing to 7) only at multiples-of-4 distances; at all other distances they sum to $6 \pmod 7$. The structural parallel between the two encodings holds in form — directional digits packed by position — but the verification guarantee is weaker for Doomyears and should not be overstated.

4 Cognitive complexity comparison

Operation	**Standard**	**Calamity Tables**
Year-offset step		
Integer division ($\lfloor y/4 \rfloor$)	1	0
Multi-digit addition	1	0
Modular reduction (mod 7, large)	1	0
Small subtraction (distance to anchor)	0	1
Table recall	0	1
Month–day step		
Subtraction (target − doomsday)	1	0
Sign correction / mod 7	1	0
Gap measurement (to month anchor)	0	1
Digit selection + single-digit add	0	1
Total operations	5	4
Operation dependency	Serial	Independent
Max intermediate value	124	6
Divisions required	1	0
Mod reductions on large numbers	2	0

Table 4: Mental operations for the two variable steps combined.

The operation count understates the difference. The standard method's operations are serial (each depends on the prior result) and involve intermediate values up to 124. The Calamity Table operations involve no value larger than 6 and the two steps (month and year) are fully independent.

5 Memorisation burden

The complete Calamity Table system requires:

- 4 century anchors (Conway's original).
- 12 vectorized doomsday codes (4 distinct non-zero pairs in two orientations, plus three zero-months).
- 16 Doomyears (or equivalently, 2×16 directional packed numbers).

Structural redundancy reduces the effective burden: month code digits sum to 7; mirror pairs (16/61, 25/52, 34/43) halve the distinct codes; the leap-year stumble rhythm makes the year sequences reconstructable from checkpoints; and Doomyear directional digits sum to 7 at multiples-of-4 distances (and to 6 otherwise), providing a partial derivation check.

6 Historical context

The 28-year periodicity is classical (Carroll [2]). Various year-offset shortcuts exist, including Conway's "odd+11" rule [1]. The month–day step has historically been handled by memorising the traditional doomsday dates and performing a subtraction.

The vectorized doomsday encoding — representing each month's doomsday information as a directionally-aware, self-verifying two-digit code derived from multiple-of-7 anchors — appears to be new. The structural parallel between the month codes and the Doomyear codes (both using complementary digit pairs, spatial direction encoding, and digit-sum verification) unifies the two variable steps of the algorithm under a single design principle — the Calamity Tables.

The classification of anchor-date systems into seven equivalence classes (Section 2.5) and the proof that Conway's class is uniquely optimal for the vectorized encoding also appear to be new. The explicit rotation cycle connecting the seven classes (Proposition 14), the identification of the month grouping partition $\{3, 2, 2, 2, 1, 1, 1\}$ as a structural fingerprint of the Gregorian calendar, the complete code table for all seven classes (Table 2), and the group-theoretic framing of the classification as a $\mathbb{Z}/7\mathbb{Z}$ action do not appear in the prior literature. Wang's null-days [5], which belong to the $k = 5$ class, confirm that the encoding technique generalises beyond Conway's specific dates, while the optimality result (Proposition 19) shows that Conway's dates yield the best code table among all seven classes.

References

[1] E. R. Berlekamp, J. H. Conway, and R. K. Guy, *Winning Ways for Your Mathematical Plays*, vol. 2, Academic Press, 1982.

[2] L. Carroll, "To find the day of the week for any given date," *Nature*, vol. 35, no. 892, 1887, p. 517.

[3] J. H. Conway, "Tomorrow is the day after doomsday," *Eureka*, vol. 36, 1973, pp. 28–31.

[4] C. Fong and M. K. Walters, "Methods for accelerating Conway's Doomsday algorithm," *ICERM Technical Report*, 2010.

[5] X.-S. Wang, "Calculating the day of the week: null-days algorithm," *arXiv preprint* arXiv:1404.2510, 2014.